MASTER BUILD PROMPT â€” AGENTIC SEO SAAS V1

You are a principal full-stack engineer, AI systems architect, product designer, DevSecOps engineer, and QA lead. Build a polished, working, secure multi-tenant **Agentic SEO SaaS V1** from this specification. This is an implementation assignment, not a request for a mockup or an architecture-only response. Work iteratively, create actual files, run tests, and document anything that cannot be completed without credentials.

0. Mission and non-negotiable outcome

Build a cloud-ready SEO operating system that lets authorized users connect a website, audit it, review AI-generated SEO recommendations, apply permitted fixes through platform connectors, verify outcomes, and monitor results. Support small businesses, SaaS teams, and agencies globally. The controlled pilot must prove an end-to-end **WordPress metadata fix on an owned test site**, including before/after preview, policy decision, actual write, live verification, audit trail, and rollback. Support Shopify and GitHub with real authorized integrations; GitHub must create pull requests, not push directly to production. Never pretend a simulated write is real.

The project must run locally in demo mode without paid API credentials, using explicitly labeled fixtures and mock connectors; production integrations must require real credentials and must never silently fall back to mock mode. No fake live metrics or fabricated ranking gains.

1. Product scope and release strategy

- V1 pilot audience: small businesses, SaaS businesses, and SEO agencies, onboarded in phases.
- Global-ready SaaS with tenant isolation, RBAC, configurable locale/time zone, and privacy controls.
- WordPress, Shopify, GitHub integrations. Public read-only auditing for other accessible sites.
- Technical SEO, on-page/content, keyword strategy, limited AI-search visibility monitoring.
- Hybrid autonomy: scheduled scans and event-driven scans; allow only explicitly authorized low-risk changes to auto-execute; sensitive changes require human approval.
- AI content is **draft only** and cannot publish without explicit editorial approval.
- Pricing model: subscriptions plus metered AI credits; implement a billing abstraction and test mode.
- Agency workspaces, client roles, and branded reports.
- First milestone: real end-to-end vertical slice, not just dashboard visuals.
- Small team (2â€“3 developers), target initial infrastructure spend $200â€“$500/month under bounded pilot usage. Do not assert that budget is guaranteed.

P0 â€” pilot-critical features

1. Account registration/login, organizations, workspace membership, roles (owner/admin/editor/analyst/viewer), tenant isolation.
2. Website registration, domain ownership verification, connector OAuth/token flows with minimum privileges, connection status.
3. Async site crawling: robots.txt, sitemap discovery, scoped URLs, canonical tags, metadata, headings, HTTP status, redirects, internal links, basic structured data, image alt, indexability signals.
4. Deterministic SEO rule engine with issue evidence, severity, affected URLs, remediation suggestions, and an **internal diagnostic health score** clearly labeled as not a Google score.
5. Four logical AI agents: Audit, Strategy, Execution, Measurement, coordinated through persisted workflows, bounded tools, and typed schemas.
6. AI recommendations and metadata drafts, evidence-linked explanations, prioritized plans.
7. Patch preview/diff, risk policy, approval inbox, immutable patch hash, authorized execution, re-read/live verification, rollback where connector supports it, immutable audit history.
8. WordPress metadata write integration (companion plugin if necessary); Shopify supported resource SEO fields; GitHub App PR creation for configured site repository.
9. Event-driven and scheduled crawls, job monitoring, retries, deduplication, budgets, kill switch.
10. Search Console OAuth integration, performance overview, delayed data handling, measurement caveats.

11. Polished dashboard, site explorer, issues, copilot, automations, approvals, execution history, settings.
12. Security protections, observability, migrations, integration tests, operational documentation.

P1 â€” finish for full V1 after vertical slice

- Internal link fixes (review first), structured data fixes (review first; validate JSON-LD and supported schema semantics).
- Keyword research using Search Console plus a pluggable paid-data provider, with source labels and budget caps.
- AI Content Studio: briefs, drafts, editorial approvals; no automatic mass publishing.
- Agency client workspaces, branded PDF reports, workspace-specific permissions.
- Subscription tiers, usage/AI credit ledger, payment provider test integration and webhook verification.
- AI-search visibility beta: limited, explicitly sampled and attributed observations; do not claim exhaustive AI search impression coverage.
- Additional site analytics, performance regressions, email/in-app notifications.

Explicit non-goals

No guaranteed rankings, unrestricted AI publishing, backlink spam, SERP scraping that violates provider terms, arbitrary remote code execution, broad robots.txt/canonical changes without approval, unreviewed bulk redirects, or unsupported website modifications.

2. Design direction â€” premium, beautiful UI

Create a visually striking but professional, production-grade SaaS. Design should feel like a premium AI operations platform, not a generic admin template. Use a distinct original visual identity with an evocative working product name (keep brand tokens easy to rename). Visual goals: calm confidence, intelligent automation, polished data density, excellent information hierarchy.

- Next.js App Router + TypeScript + Tailwind CSS + shadcn/ui + Lucide icons.
- Default theme: sophisticated light mode with warm off-white canvas, charcoal typography, restrained electric indigo/teal accents; equally polished dark mode with near-black graphite backgrounds. Use CSS variables/design tokens, accessible contrast, consistent spacing and radii.
- Typography: expressive display headings, highly legible UI/body typography, tabular numerals for metrics; use freely available web-safe or open-license font delivery (do not bundle proprietary fonts).
- Layout: collapsible sidebar with clear navigation, topbar with site selector, global search, notifications, profile, contextual breadcrumbs; responsive tablet/mobile layouts.
- Motion: subtle Framer Motion transitions, skeleton loading, hover states, optimistic feedback where safe, tasteful page entrances. Respect `prefers-reduced-motion`. Avoid excessive parallax in operational screens; optional restrained landing-page depth effects.
- Visuals: custom charts (Recharts), trend sparklines, severity distributions, workflow timeline, before/after diffs, URL inspection panels, interactive site map or page relationship graph if useful.
- Use real component states: loading, empty, success, warning, error, offline/connector disconnected, approval pending, execution failed, rollback available. Never render inert decorative controls.
- Include a polished public marketing landing page: hero with compelling value proposition, how-it-works sequence, supported platforms, security/approval story, use cases, pricing teaser, FAQs, CTA, footer. Do not make unverifiable marketing claims.
- Build a design system: buttons, inputs, data tables, filters, modals, command palette, badges, alerts, toasts, drawers, tabs, cards, charts, stepper, empty states, audit timeline, and risk/approval components.
- Responsive behavior, keyboard accessibility, focus rings, semantic labels, WCAG AA-oriented contrast and usability.
- UI should be filled with clearly marked *demo data* in demo mode and real API data when connected. Do not invent customer performance in production.

Required screens and UX flows

1. Marketing landing / pricing / sign-in.
2. New organization and agency workspace onboarding.

3. Add website wizard: domain -> ownership -> platform connector -> permissions -> crawl settings -> first audit.
4. Overview: site health, issues, top priorities, traffic trends, automation activity, recent changes.
5. Site explorer: searchable/filterable URLs, page-level SEO diagnostics, crawl snapshots.
6. Audit center: issue categories, severity, evidence, affected pages, remediation details.
7. AI Copilot: grounded Q&A over the selected site's evidence, action proposal cards, clear limitations.
8. Strategy / keyword intelligence: query groups, content opportunities, data source provenance.
9. Content Studio: brief, draft, side-by-side preview, editorial review queue.
10. Auto-Fix center: before/after diffs, connector capabilities, risk score, policy, dry run, execute.
11. Approval inbox: approve/reject with reason; immutable patch version and audit record.
12. Automation center: schedules, event triggers, policy modes (Observe/Assist/Autopilot), runs, pause/kill switch.
13. Analytics: Search Console metrics, date range, annotations for deployments, honest attribution notes.
14. Agency client workspaces: sites, role assignments, branded reports.
15. Billing/usage: plan, AI credits, quotas, ledger, invoices where provider is connected.
16. Settings: integrations, secrets status, roles, notifications, retention, data export/deletion.

3. Architecture and repository structure

Use a modular monolith with separate background workers. Prefer a monorepo:

```text
agentic-seo/
  apps/web/                # Next.js UI
  apps/api/                # FastAPI API
  apps/worker/             # Celery worker entrypoints
  packages/ui/             # Shared UI tokens/components (if needed)
  packages/contracts/      # OpenAPI-generated TS client / schemas
  integrations/wordpress/  # Companion WordPress plugin
  infra/                   # Docker Compose and optional AWS IaC
  docs/                    # Architecture, security, setup, API, operations
  tests/                   # E2E and integration suites
  .env.example
  docker-compose.yml
  README.md
```

Backend modules: auth, tenancy, workspaces, sites, connectors, crawler, seo_rules, findings, ai_gateway, agents, plans, change_sets, policy, approvals, executions, schedules, analytics, keywords, content, reporting, billing, audit, notifications.

Stack: FastAPI, Pydantic v2, SQLAlchemy 2, Alembic, PostgreSQL 16+, Redis, Celery, HTTPX, selectolax/lxml, Playwright only when required, S3-compatible object storage, structured JSON logging, OpenTelemetry, Sentry optional. AI via configurable managed LLM provider adapter initially, with a private-model adapter interface. Use LangGraph only if it materially helps persisted workflows; a custom typed state machine is acceptable. Local development via Docker Compose; AWS ECS/Fargate or comparable managed deployment documented for pilot. Keep infrastructure cost bounded.

4. Domain model and database

Create real Alembic migrations and ORM models for:

- tenants, users, memberships, roles/permissions
- client_workspaces, workspace_memberships
- sites, site_verifications, connectors, connector_capabilities, connector_secret_refs
- crawl_jobs, pages, page_snapshots, crawl_edges
- seo_rules, seo_findings, finding_occurrences
- keyword_queries, keyword_clusters, data_source_syncs
- action_plans, plan_items, content_drafts

- change_sets, patch_versions, approvals, execution_runs, rollback_snapshots
- automation_policies, automation_schedules, event_inbox, workflow_runs, agent_runs, tool_invocations
- search_metrics, measurement_annotations, reports
- plans, subscriptions, credit_ledger, usage_records
- audit_events, notifications

Requirements: UUID IDs; created_at/updated_at UTC; proper foreign keys; composite tenant-scoped constraints; uniqueness for normalized URLs per site; indexes for crawl/findings/status queries; JSONB for evidence with schema validation; append-only audit records; retention/deletion workflows; PostgreSQL RLS for tenant-owned tables where appropriate plus service-layer checks. Do not accept tenant IDs as authority from request bodies. Use a transactional outbox or equivalent reliable handoff for side-effecting jobs.

5. Core APIs â€” define OpenAPI schemas and implement

All routes under `/api/v1`, authenticated unless explicitly public. Use cursor pagination for large lists; structured errors; rate limits; idempotency keys for writes; CSRF-safe browser auth; connector OAuth state/PKCE as appropriate.

```text
POST   /organizations
GET    /organizations/current
POST   /workspaces
GET    /workspaces
POST   /sites
GET    /sites
GET    /sites/{site_id}
POST   /sites/{site_id}/verification/start
POST   /sites/{site_id}/verification/check
POST   /integrations/{provider}/authorize
GET    /integrations/{provider}/callback
GET    /sites/{site_id}/connectors
POST   /sites/{site_id}/crawls
GET    /crawls/{crawl_id}
GET    /sites/{site_id}/pages
GET    /sites/{site_id}/findings
GET    /findings/{finding_id}
POST   /sites/{site_id}/plans
GET    /plans/{plan_id}
POST   /change-sets
GET    /change-sets/{change_id}
POST   /change-sets/{change_id}/validate
POST   /change-sets/{change_id}/approve
POST   /change-sets/{change_id}/reject
POST   /change-sets/{change_id}/execute
POST   /change-sets/{change_id}/rollback
GET    /executions/{execution_id}
POST   /copilot/messages
GET    /sites/{site_id}/keywords
GET    /sites/{site_id}/performance
GET    /sites/{site_id}/automations
PUT    /sites/{site_id}/automations/policy
GET    /usage
GET    /reports
POST   /reports
GET    /audit-events
POST   /webhooks/{provider}
```

For each endpoint, implement Pydantic request/response contracts, RBAC, input validation, examples, error codes, and tests. Use `202 Accepted` for queued jobs. Do not let `execute` accept arbitrary new patch contents. Approval must bind to exact immutable patch hash and site/version. An approval is invalidated if patch content changes.

6. SEO crawler and rule engine

Crawler must:

- Normalize/canonicalize URLs correctly; enforce same-site and configurable path scope.
- Honor robots directives where applicable, site ownership, crawl-delay/polite throttling, per-host concurrency, size/time/page caps, and retry limits.
- Prevent SSRF: reject private/link-local/loopback/metadata IPs, validate DNS on each connection/redirect, restrict schemes and ports, prevent DNS rebinding and network egress to internal resources. Isolate browser rendering workers.
- Parse robots.txt, sitemap XML (including sitemap indexes), HTML head metadata, heading hierarchy, canonical and robots meta, status codes, redirects, links, JSON-LD, images and alt text.
- Use JS rendering only for pages that need it; cap browser sessions.
- Store page snapshots and source evidence; track crawl changes by content hash.
- Detect missing/duplicate titles/descriptions, broken internal links, noindex/canonical inconsistencies, redirect chains, malformed structured data, H1 problems, missing alt, sitemap mismatch, and page performance signals from approved APIs.
- Clearly distinguish deterministic errors from heuristic opportunities (e.g., 'thin content').
- Score health with a documented internal weighting, not a Google ranking score.
- Provide fixture-based crawler and rule tests.

7. Four agent contracts

Implement **logical** agents with structured I/O and bounded tools:

1. **Audit Agent** â€” consumes snapshots and rule findings; produces evidence-grounded explanations and issue clusters. Tools: get_page_snapshot, list_findings, validate_schema, inspect_sitemap.
2. **Strategy Agent** â€” consumes findings, Search Console query data, business goals, previous changes; produces prioritized action plans, keyword opportunities and content drafts. Tools: get_search_queries, get_findings, cluster_keywords, propose_metadata, draft_content.
3. **Execution Agent** â€” consumes immutable proposed patches and capability/policy results; may call prepare_patch, validate_patch, request_approval, apply_authorized_patch, verify_change, rollback. Never bypasses policy engine or executes arbitrary LLM-generated code.
4. **Measurement Agent** â€” consumes execution records and GSC metrics; produces verified technical outcomes, trend analysis and regression warnings. Tools: get_gsc_metrics, compare_periods, get_execution_history, schedule_crawl.

Agent outputs must be Pydantic-validated, cite internal evidence IDs, record prompt/model version and token/cost usage, use timeouts/retries, and refuse unsupported claims. Crawled pages are untrusted data, never agent instructions. No unrestricted browsing, shell, secrets, or cross-tenant retrieval tools. AI-generated confidence alone cannot authorize a production write.

8. Connector interface and implementation

Define a typed interface with:

```python
class WebsiteConnector(Protocol):
    async def verify_ownership(self, site): ...
    async def capabilities(self, site): ...
    async def discover_pages(self, site): ...
    async def read_resource(self, resource): ...
    async def prepare_patch(self, change): ...
    async def apply_patch(self, approved_change): ...
    async def verify_patch(self, execution): ...
    async def rollback(self, execution): ...
```

Each connector reports supported actions, preconditions, permissions, and whether rollback is supported. Unsupported writes remain recommendations.

- **WordPress**: authenticated WordPress API plus small companion plugin for reliable SEO metadata access, revisions, least-privilege permissions, and safe restoration; do not assume all SEO plugins expose identical fields.
- **Shopify**: authorized app and supported Admin GraphQL API resources; begin with supported product/page SEO metadata; separate theme edits and require review; respect rate limits and app scopes.

- **GitHub**: GitHub App installation with limited repository permissions; configured framework-specific file mapping; create branch, commit, PR with diff and checks; never direct push to default branch. A PR is not a live deployment. Verify after merge and deployment signal.
- **Public-only**: read-only crawl and findings for other websites; no claims of write access.

Production connector credentials stored by reference in a managed secrets store. Encrypt, rotate and revoke. No secrets in logs, source, AI prompts or audit event payloads.

9. Policy and safe execution

Modes: Observe, Assist (default), Autopilot (explicit opt-in). Automatic edits allowed only for specific whitelisted action types and site scopes, after connector capability, tenant permission, risk limits, deterministic validation, snapshot/recovery checks and conflict locks pass. Default metadata publishing to approval until policy explicitly authorizes it. Internal link and structured data edits require review initially; AI article publishing always requires editorial approval in V1. Broad canonical, robots, redirect, deletion or bulk edits restricted.

Workflow:
`finding -> proposal -> validation -> immutable patch -> policy evaluation -> approval if required -> queued execution -> connector write -> verify resource/live page -> completed or failed -> rollback where supported -> measurement`.

Implement idempotency, optimistic concurrency/version checks, distributed locks, bounded retries, dead-letter handling, before/after diff, audit log, execution cancellation where safe, emergency kill switch, manual rollback, alerting and recovery tests. Show a clear warning when rollback is not guaranteed or requires human action. No automatic change to unverified websites.

10. Data integrations and SEO analytics

- Google Search Console OAuth and Search Analytics for clicks, impressions, CTR, average position, page/query/date. Treat API data as delayed and limited; do not assert exhaustive coverage. URL Inspection reports known indexed status, not guaranteed indexing.
- Google PageSpeed Insights / CrUX where available; distinguish lab vs field metrics.
- GA4 optional P1 connector with clearly labeled traffic/conversion metrics.
- Keyword provider abstraction: free GSC source plus paid provider with quotas, caching and explicit provenance; no fake search volume numbers.
- AI-search visibility beta: labeled samples with timestamp, source, geography and limitations; no blanket claims about LLM rankings or citations.
- Measure technical fix verification separately from organic search outcomes. Include date annotations, before/after comparisons, confidence caveats and seasonal/external-factor disclaimers.

11. Billing and agencies

Implement agency organizations with multiple client workspaces, per-workspace site access, role templates, branded reports, and secure client boundaries. Implement plans, subscriptions, quota enforcement, append-only AI credit ledger, usage reconciliation, test payment flows and verified webhooks. Avoid hardcoded billing amounts; seed editable example plans. All expensive operations must check available budgets before scheduling and charge exactly once using idempotency keys.

12. Test and acceptance requirements

Required automated tests:
- Unit tests for SEO rules, URL normalization, policy engine, scoring, credit accounting, schema validation.
- API contract tests, tenant isolation and RBAC tests, invalid approval/patch replay tests.
- Integration tests with WordPress test fixture/plugin, Shopify sandbox or test store, GitHub test repository and webhook fixtures.
- E2E Playwright UI tests for onboarding, audit, recommendation, approval, execution, verification, rollback, agency boundaries.
- Security tests for SSRF, prompt injection, webhook replay, secret leakage, IDOR, cross-tenant retrieval, unapproved writes.

- Failure injection: timeout, connector 429/500, worker restart, duplicate job, stale resource version, deployment not yet live.
- Accessibility and responsive checks; typecheck, lint, unit, integration and E2E CI.

M1 acceptance: on an owned WordPress staging site: register/verify -> connect -> crawl -> detect missing description -> AI or deterministic fallback generates a draft -> review/policy -> write -> verify actual live HTML -> display before/after + audit trail -> restore previous value. No demo substitution in production mode.

V1 pilot acceptance: WordPress M1, supported Shopify SEO update with verification, GitHub PR workflow, working agency permissions/report, GSC integration, billing metering, safety checks and observability. All must be demonstrated against authorized test resources. Define measurable pilot SLOs and document load test results rather than claiming guaranteed numbers.

13. Delivery order and working method

Work in vertical slices, commit coherent changes, and keep the app runnable throughout:

Phase A â€" foundations (weeks 1â€"2): repo, design system, marketing page, app shell, auth, tenant models, Docker Compose, database migrations, seed fixtures, API client.

Phase B â€" real first fix (weeks 3â€"4): scoped crawler, SEO rules, WordPress connector/plugin, metadata draft, immutable change sets, approval/policy, actual write, verification, rollback, polished issue/fix UI.

Phase C â€" agentic workflow (weeks 5â€"6): AI provider gateway, four agent contracts, persisted orchestration, copilot, workflow history, scheduling, events, budgets.

Phase D â€" multi-platform (weeks 7â€"10): Shopify write path, GitHub PR path, internal link and structured data review workflows, end-to-end integration tests.

Phase E â€" intelligence and agency (weeks 11â€"13): Search Console, keywords, content studio, agency workspaces, reports, AI visibility beta.

Phase F â€" pilot hardening (weeks 14â€"16): billing and credits, security hardening, monitoring, CI, backup/restore tests, performance testing, docs and pilot checklist.

If time is constrained, finish and test Phase B before adding broad P1 features. Do not replace functionality with decorative mocks. If credentials or external resources are missing, implement a fully labeled demo adapter, complete integration code behind configuration, and provide precise setup and verification instructions.

14. Developer deliverables

Create:
1. Runnable monorepo and complete source code.
2. `.env.example` with documented required and optional variables (never real secrets).
3. Docker Compose for API, web, Postgres, Redis, worker and optional local object storage.
4. Database migrations and seed data.
5. WordPress companion plugin source and install guide.
6. Shopify app setup guide and connector implementation.
7. GitHub App setup guide and PR integration.
8. Google Search Console OAuth setup guide.
9. API OpenAPI documentation and generated/typed frontend client.
10. Architecture decision records, threat model, connector capability matrix, and runbooks.
11. Unit, integration, security and E2E tests with commands.
12. README: one-command local start, demo login/test fixtures, real connector setup, production deployment notes.
13. Pilot checklist and a truthful completion matrix (`implemented`, `tested`, `requires credentials`, `deferred`).

15. Execution instructions for the coding agent

First inspect the current repository. If empty, scaffold the monorepo. Provide a short

architecture/build plan and immediately implement Phase A and the Phase B vertical slice. Do not ask open-ended product questions; use the decisions above. If external credentials are necessary, build the integration and supply exact setup steps while continuing other work. Execute terminal commands, migrations, tests and builds when your environment permits. Fix errors before moving forward. Do not claim tests passed without running them. Do not claim external websites were modified without verified connector results. Keep UI and backend connected with real typed APIs; demo mode must be visibly labeled. At the end of each milestone report files created, working flows, test results, blockers and next actions.

Final product quality bar: beautiful, fast, accessible UI; real SEO intelligence; controlled agentic automation; verifiable website fixes; secure tenant separation; credible operations and developer documentation.